Computer Architecture Today

Informing the broad computing community about current activities, advances and future directions in computer architecture.

DNN Accelerator Architecture – SIMD or Systolic?

by Reetuparna Das and Tushar Krishna on Sep 17, 2018 | Tags: Accelerators, Machine Learning, Specialization

While the concept of hardware acceleration has been around for a while, DNN accelerators are perhaps the first to see the light of commercial adoption due to the AI/ML wave. Giant software corporations, veteran hardware companies, and a plethora of start-ups have invested in DNN acceleration. Commercial examples can be categorized into tightly-coupled 2D systolic-arrays (Google's TPU, ARM's project Trillium), loosely coupled spatial arrays with independent PEs (WaveComputing's DPU), ultra-wide SIMD (Microsoft Brainwave, NVDLA), and SIMT with or without specialized CISC operations (e.g. GPUs w or w/o tensor cores). Furthermore, tensor cores themselves are tiny systolic arrays, while loosely coupled spatial arrays can be operated as full MIMD units or multiple SIMD units.

Thus, in this blog we take the liberty to compare a “generalized” 2D systolic array accelerator for matrix multiplication and a “generalized” 1D SIMD. We compare the *architectures* across various qualitative metrics, not on the implementation platform through which they may be deployed (ASIC/FPGA/PIM).

Disclaimer: This is a pure academic exercise which is not endorsing any particular commercial product.

Compute Density. Winner: <Systolic>

If we are just looking at raw MACs per unit area dedicated to compute, systolic architecture seems like a clear win. A clean systolic data-movement needs data to enter the array from specific entry points, after which the PE’s communicate locally in an established rhythm. The individual PEs in the array do not require any special instructions. SIMD, suffers from dedicated structures for data delivery and instruction broadcasting. While this is apples vs oranges, TPU has higher TOPs/mm^2 than GPUs. TPU v1 is 90 TOPs for less than ~330 mm^2 @28nm, and Volta is 125 TOPs for ~800 mm^2 @12nm (12nm is two technology nodes below 28nm).

Distribution Bandwidth. Winner: <Systolic>

A MxN Systolic array needs (M+N) external channels to feed new data to the array every cycle from the edges. O(MN) neighbor-neighbor connections increase the internal bandwidth available for data distribution. In contrast, a SIMD or spatial design with MxN PEs needs a separate data distribution fabric (buses or trees) with support for multicasts and high bandwidth to keep the MxN PEs fed to reduce stalls.

Reduction. Winner: <Open>

DNN algorithms require reduction across large number of intermediate/partial results. Systolic arrays perform *spatio-temporal* reduction by forwarding and reducing partial sums along a row or column. Thus reduction is natural for systolic arrays and has low implementation complexity. Spatial SIMD designs may perform spatio-temporal reduction like the systolic array, or spatial reduction using explicit reduction trees (e.g., Brainwave, NVDLA) or purely in-place temporal reduction. This enables more flexible mapping strategies. To complete the picture, GPU’s do not have an explicit reduction network. Reduction can proceed temporally as a parallel prefix sum which requires logarithmic number of steps and bunch of intermediate register reads/writes.

Compute Unit Utilization. Winner: <SIMD>

The 2D nature of currently deployed systolic arrays makes them extremely efficient for matrix-matrix (M*M) multiplication, but not super-efficient at other operations such as Matrix-Vector (M*V), or Vector-Vector (V*V). SIMD-style designs meanwhile can work efficiently for M*V and V*V. Convolutions can be transformed into M*M and mapped efficiently over systolic arrays. However, the fixed sized dimensions can lead to under-utilization for matrices whose dimensions do not map perfectly to the array dimensions. The extreme case of this is M*V computations used

heavily by LSTMs and MLPs that lead to under-utilization in systolic arrays. TPU v1 reports ~60% of utilization for compute cycles for a benchmarked LSTM and ~50% for another benchmarked CNN. Overall, the utilization is ~6% for LSTMs, but this includes all stall cycles (including data loading, which is incurred for both architectures).

Latency and Throughput and Batching. Winner: <Training-Tie, Inference-SIMD>

The latency and throughput of each design is a direct function of its compute unit utilization.

Systolic-arrays work well for DNNs where M*M operations is the intrinsic operation (e.g., CNNs), but have low utilization for those that use M*V operations (e.g., RNNs). This argument was made by BrainWave, and proven quantitatively for LSTMs. During training, the availability of mini-batches can amortize weight loading time over *multiple inputs* in both designs – i.e., create a M*M computation. However, during inference, batching may not be possible as requests arrive serially. In such scenarios, the compute unit utilization of systolic arrays is highly dependent on the array's aspect ratio and vector dimensions. The 1D SIMD category is more versatile, since M*M can always be temporally unfolded into multiple M*V and V*V ops, but not vice versa. Moreover, with the recent trend in DNNs towards smaller 1×1 and 3×3 weight kernels, SIMD might be more future proof for inference.

Flexibility/Programmability. Winner: <SIMD>

2D systolic arrays operate via a rigid neighbor-to-neighbor forwarding mechanism of inputs/weights/partial sums every cycle. Individual MACs cannot stall – if there is insufficient bandwidth, the entire array needs to stall. Spatial SIMD designs can allow individual PEs to stall, enabling higher flexibility in terms of work distribution and mapping. Depending on the reduction mechanism supported (temporal or spatial or spatio-temporal), mapping strategies can also be varied depending on the layer dimensions. This is especially important for the ML field as it continues to evolve rapidly.

Reuse. Winner: <Tie>

Qualitatively, this one is a tie. Reuse depends on the amount of on-chip data storage, and the dataflow being employed. Both architectures can keep weights or inputs stationary at the MACs to exploit *temporal reuse*. Both designs also provide *spatial reuse* by multicasting weights or inputs – systolic arrays do it via store-and-forward while SIMD employs buses or trees. And finally, reduction of partial sums for accumulation exploits *spatio-temporal* reuse in both designs.

Sparsity. Winner: <SIMD/Open>

Sparsity is a challenge for both the architectures and currently an open-research problem. Systolic arrays need a rhythmic data flow, and zero's in random positions cannot be removed for enhancing utilization. SIMD, on the other hand, can in principle support mechanisms to detect zeros and only map non-zero weights/inputs. But it would suffer from divergence and unequal

work distribution. Structured sparsity (where non-zero's can be clustered in some fashion) might be easier to leverage in a 1D SIMD architecture vs 2D systolic.

Complexity. Winner: <Systolic>

Hardware complexity is always subjective. However, systolic arrays are inherently easier to understand and operate due to simpler instruction/data supply mechanisms and simpler synchronization – the entire array stalls or operates. This also simplifies control circuitry. Naturally, this comes at the cost of flexibility, as discussed before.

Data Loading & Integration with Host. Winner: <Open>

Unfortunately, data loading and integration with a host is a problem for all accelerators, not just DNN accelerators. The bandwidth requirement from external memory would depend on the dataflow and reuse characteristics that the accelerator supports. Dense in-memory architectures that can increase *both* compute/mm^2 and storage/ mm^2 by merging the two, maybe a promising solution towards this problem

In conclusion, 2D systolic arrays are great for their compute density and simplicity but may suffer from utilization across DNN models. Spatial SIMD architectures can provide more flexibility and performance but at the cost of more expensive control and interconnect circuitry. Energy-efficiency, which depends on both runtime (i.e., utilization and stalls) and power (i.e., control and complexity) will depend on the DNN kernel characteristics and deployment scenarios. Architectures that can marry the benefits of both may be a promising direction for DNN accelerators.

Acknowledgments: *Thanks to Michael Pellauer from NVIDIA, Ananda Samajdar and Hyoukjun Kwon from Georgia Tech, Charles Eckert and Xiaowei Wang from Michigan for their feedback.*

About the Authors: *Reetuparna Das is an Assistant Professor at the University of Michigan.Tushar Krishna is an Assistant Professor at Georgia Tech. Feel free to contact them at reetudas@umich.edu and tushar@ece.gatech.edu.*

Share this:

Facebook Twitter LinkedIn

Comments Community Privacy Policy 1 Login

Recommend 4 Tweet Share Sort by Newest

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

SUNIL • 2 years ago • edited

Can suggest , any compiler for systolic array based acceleration

• Reply • Share ›

Xiaohu Tang SUNIL • a year ago

CONTRIBUTE

Editor: Rajeev Balasubramonian

Associate Editor: Christina Delimitrou

Contribute to Computer Architecture Today

RECENT BLOG POSTS

Data Engineering for Everyone

From the Editors' Desk — 2021 Edition

Questions About Policies & Processes in the Wake of JIC

Valuing Diversity, Equity, and Inclusion in Our Computing Community

Languages, Tools, and Techniques for Accelerator Design

ARCHIVES

March 2021 (1)

February 2021 (8)
January 2021 (2)
December 2020 (5)
November 2020 (5)
October 2020 (6)
September 2020 (3)
August 2020 (2)
July 2020 (4)
June 2020 (5)
May 2020 (6)
April 2020 (4)
March 2020 (6)
February 2020 (2)
January 2020 (2)
December 2019 (3)
November 2019 (6)
October 2019 (3)
September 2019 (5)
August 2019 (6)
July 2019 (5)
June 2019 (4)
May 2019 (5)
April 2019 (5)
March 2019 (4)
February 2019 (5)
January 2019 (4)
December 2018 (3)
November 2018 (3)
October 2018 (4)
September 2018 (4)
August 2018 (4)
July 2018 (6)
June 2018 (4)
May 2018 (7)
April 2018 (7)
March 2018 (6)
February 2018 (4)
January 2018 (7)
December 2017 (4)
November 2017 (4)
October 2017 (6)
September 2017 (6)
August 2017 (2)
July 2017 (4)
June 2017 (6)
May 2017 (6)
April 2017 (4)
March 2017 (2)

TAGS

Academia Accelerators ACM SIGARCH Advice Architecture Benchmarks Cloud computing Conference Databases Datacenters Distributed Systems Diversity Emerging Technology FPGA Harassment Hardware Industry Interview ISCA Machine Learning Measurements Memory Mentoring Methodology Mobile Near Data Computing Networking non-volatile Operating Systems Opinion Parallelism People Performance Persistent Policy Programmability Quantum Computing Reviewing Security Specialization Storage Systems Travel Virtual Meetings Vision

SUBSCRIBE

RSS SUBSCRIBE

Get E-Mail Alerts, enter your Email:

SUBSCRIBE »

JOIN US

Keep up-to-date with the latest technical developments, network with colleagues outside your workplace and get cutting-edge information, focused resources and unparalleled forums for discussions. Learn more...

ACM SIGARCH

SIGARCH serves a unique community of computer professionals working on the forefront of computer design in both industry and academia. It is ACM's primary forum to interchange ideas about tomorrow's hardware and its interactions with software.

ACM SIGARCH

SIGARCH serves a unique community of computer professionals working on the forefront of computer design in both industry and academia. It is ACM's primary forum to interchange ideas about tomorrow's hardware and its interactions with software.

About the Site

This site is maintained by volunteers working in many programs of ACM SIGARCH. We thank you for visiting! If you have questions about the site, please send a note to our content editor.